Mini face detect and recognize interface

Amendment Record

Version	Contributor	Date	Change Log
v1.0	吴仕意	18/08/09	add description of interface

SDK

- Entity class structure

```
public class FaceInfo implements Parcelable {
/**
* The id of person which is registered, it will be null without registration.
*
*/
private String id;
/**
* The name of person which is registered
*
*/
private String name;
/**
* The head of the registered person has not yet been activated.
*
*/
private String avtar;
```

```
/**
* Time of registered
*
*/
private long time;
/**
* The pitch of the face, range:[-30,30]
*
*/
private float pitch = 0f; //角度 pitch 上下偏移:[-30,30]
/**
* The yaw of the face, range:[-30,30]
*
*/
private float yaw= 0f; //角度 yaw 左右偏移:[-30,30]
/**
*The roll of the face,range:[-180,180]
*
*/
private float roll= 0f; //角度 roll:平面旋转[-180,180]
/**
* Is it a famous
*
*/
private boolean famous;
/**
* The gender of the face,range: (female)0~100(man)（only for faceAnalyze interface）
*
*/
private int gender; //性别 (女)0~100(男)（仅限人脸分析接口）
/**
* The age of the face （only for faceAnalyze interface）
*
*/
private int age; //年龄（仅限人脸分析接口）
/**
* The X coordinates of the upper left corner of the face frame.
```

```
* The origin of the coordinates is the upper left corner of the picture.
*
* @return int
*/
private int x;

/**
* The Y coordinates of the upper left corner of the face frame.
* The origin of the coordinates is the upper left corner of the picture.
*
* @return int
*/
private int y;
/**
* Face width
*
*/
private int width;
/**
* Face height
*
*/
private int height;
/**
* The width of the preview picture
*
*/
private int imgWidth;
/**
* The height of the preview picture
*
*/
private int imgHeight;

/**
* Is it a stranger(who without registered)
*
```

```
* @return true if it is a stranger
*/
public boolean isStranger() {
    return TextUtils.isEmpty(id);
}
}
```

- java interface

```
public class FaceApi {
/**
 * Check the robot currently in the process of registering
 *
 * @param responseListener Result callback
 */
public void checkFaceInsertState(final ResponseListener<Void> responseListener);

/**
 * to start face register
 *
 * @param invokeId        invoker id, can be uuid or null
 * @param name            The name of face to be registered
 * @param responseListener Result callback, with faceId
 */
public void apiFaceRegister(final String invokeId,final String name, final ResponseListener<String> responseListener) ;

/**
* to stop face register
*
* @param userId          invoker id, only the same invokeId can stop de processing
* @param responseListener Result callback
*/
public void apiStopRegister(String invokeId,final ResponseListener<Void> responseListener);

/**
```

```
     * update the name of face
     *
     * @param id              the id of the face which to be updated
     * @param name            the new name of face
     * @param responseListener result callback
     */
    public void update(final String id, final String name, final ResponseListener<Void> responseListener);

    /**
     * delete of face info
     *
     * @param ids             the id list to be deleted
     * @param responseListener result callback
     */
    public void delete(final ArrayList<String> ids, final ResponseListener<Void> responseListener)

    /**
     * get the registered face
     *
     * @param page            pageNum
     * @param pageSize        pageSize
     * @param responseListener result callback
     */
    public void query(final int page, final int pageSize, final ResponseListener<ArrayList<FaceInfo>> responseListener)

    /**
     * face detect, detect face count.
     *
     * @param timeout         The timeout time of the face is not detected, in seconds
     * @param responseListener Result callback
     */
    public void faceDetect(long timeout, final ResponseListener<List<FaceInfo>> responseListener)

    /**
     * Face analysis，analyze the age and gender
     *
     * @param timeout         The timeout time of the face is not detected, in seconds
```

```
     * @param responseListener Result callback
     */
    public void faceAnalyze(long timeout, final ResponseListener<List<FaceInfo>> responseListener)

    /**
     * Face Recognize，recognize the face is registered or not
     *
     * @param timeout          The timeout time of the face is not detected, in seconds
     * @param responseListener Result callback
     */
    public void faceRecognize(long timeout, final ResponseListener<List<FaceInfo>> responseListener)

    /**
     * Move the robot¬s to search the face and return the face detected
     *
     * @param timeout          The timeout time of the processing, in seconds
     * @param responseListener result callback
     */
    public void findFace(long timeout, final FaceFindListener responseListener)

    /**
     * Pause to search face
     *
     * @param responseListener result callback
     */
    public void pauseFindFace(final ResponseListener<Void> responseListener)

    /**
     * Stop the search processing
     *
     * @param responseListener result callback
     */
    public void stopFindFace(final ResponseListener<Void> responseListener)

    /**
    * face track and recognize, track the face and return face info when face count change
    *
```

```
* @param timeout   The timeout time of the processing, in seconds
* @param needTrack Track the face or not
* @param faceTrackListener Result callback
*/
public void apiFaceTrack(long timeout, boolean needTrack,final FaceTrackListener faceTrackListener) {

/**
 * Stop face track processing
 *
 * @param responseListener Result callback
 */
public void apiStopFaceTrack(final ResponseListener<Void> responseListener)

public interface FaceTrackListener {
/**
* on face track start.
*/
public void onStart();
/**
* on face count change.
*/
public void onFaceChange(List<FaceInfo> faceInfos);
/**
* on process stop.
*/
public void onStop();
/**
* on fail.
*/
public void onFail(int code, String msg);

}

public interface FaceFindListener extends FaceTrackListener {
/**
* on process pause.
*/
```

```
public void onPause();

}
}
```